PROJECT CODING STAGE REPORT

Name: Chantal Pande.

ID: 667020.

Project name: Ex-change: Community skill exchange platform.

Project Overview

Project goal and scope

The goal of Ex-change is to develop a web-based application that enables individuals to exchange skills without monetary transactions. The system connects users who possess different skills and allows them to collaborate, learn and share knowledge in a peer-to-peer environment.

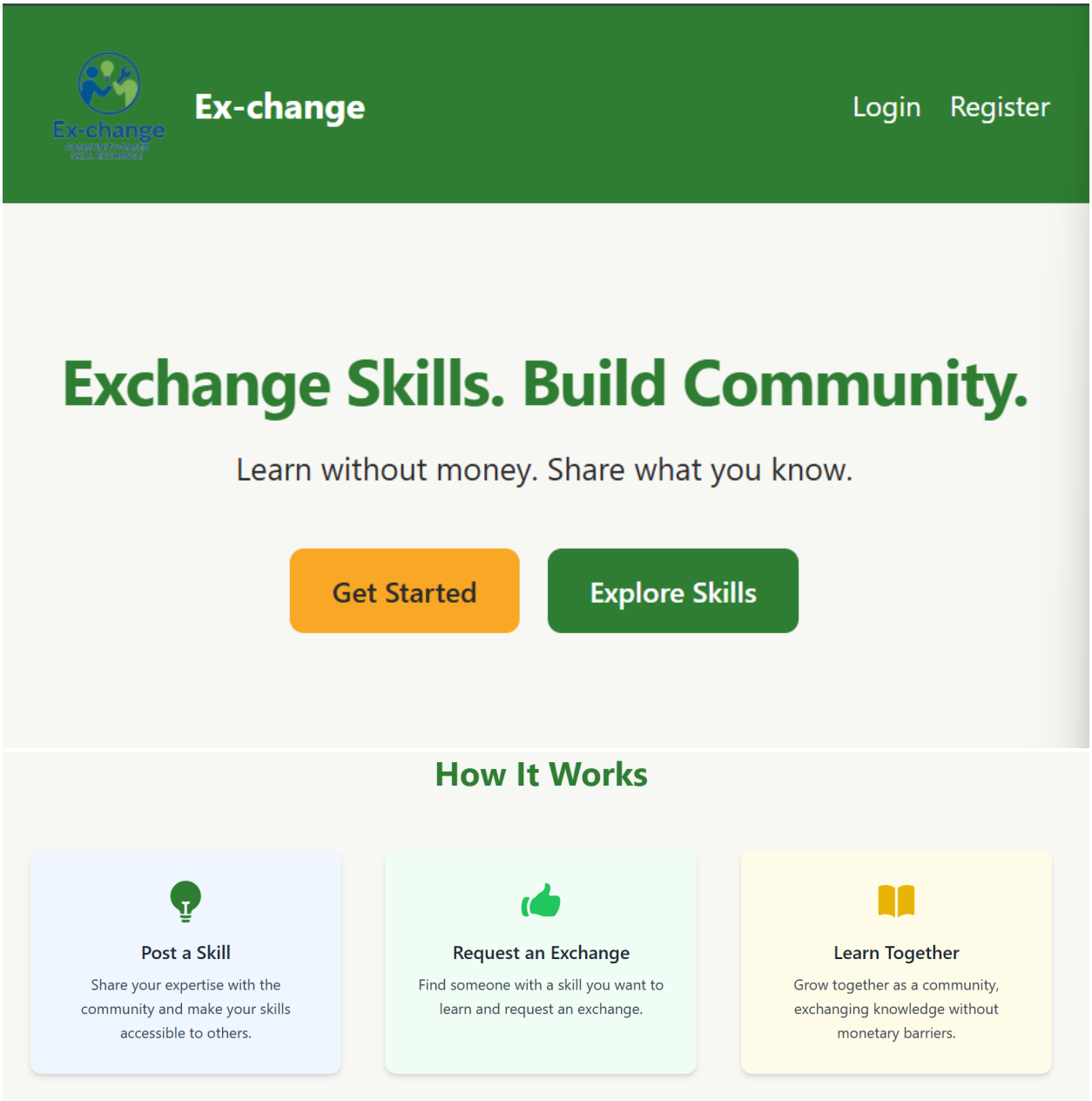
Key features planned for the final product

1. User authentication - secure registration and login using JWT authentication.
2. Skill listings - users can add skills they can offer or request.
3. Skill exchange requests - users can send and manage exchange requests.
4. Messaging systems - users can communicate within the platform.
5. Rating and review system - users rate each other after completing exchanges.
6. AI-based skill matching (Future enhancement) - the system will suggest possible skill matches between users.

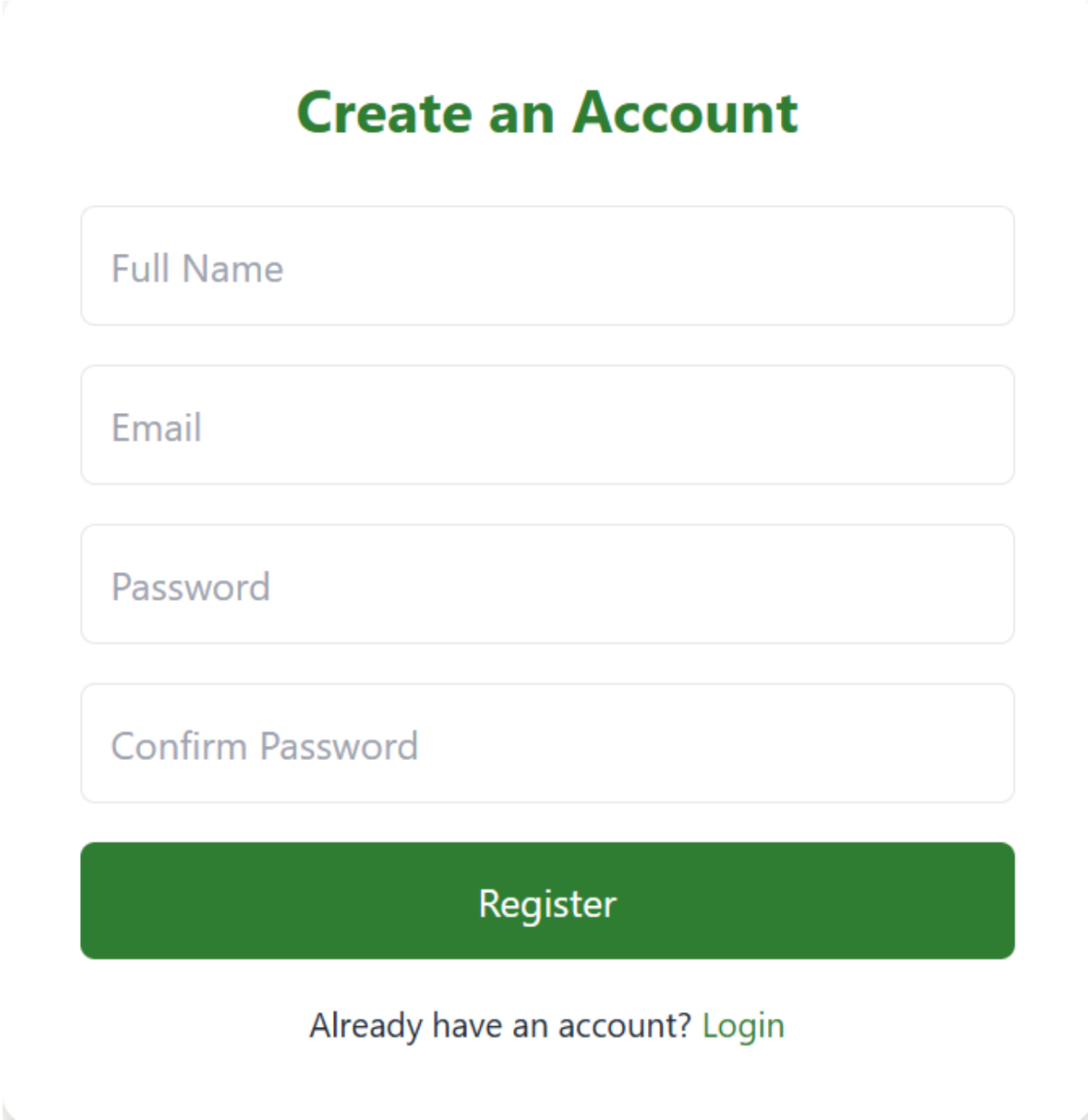
Development progress

Components implemented

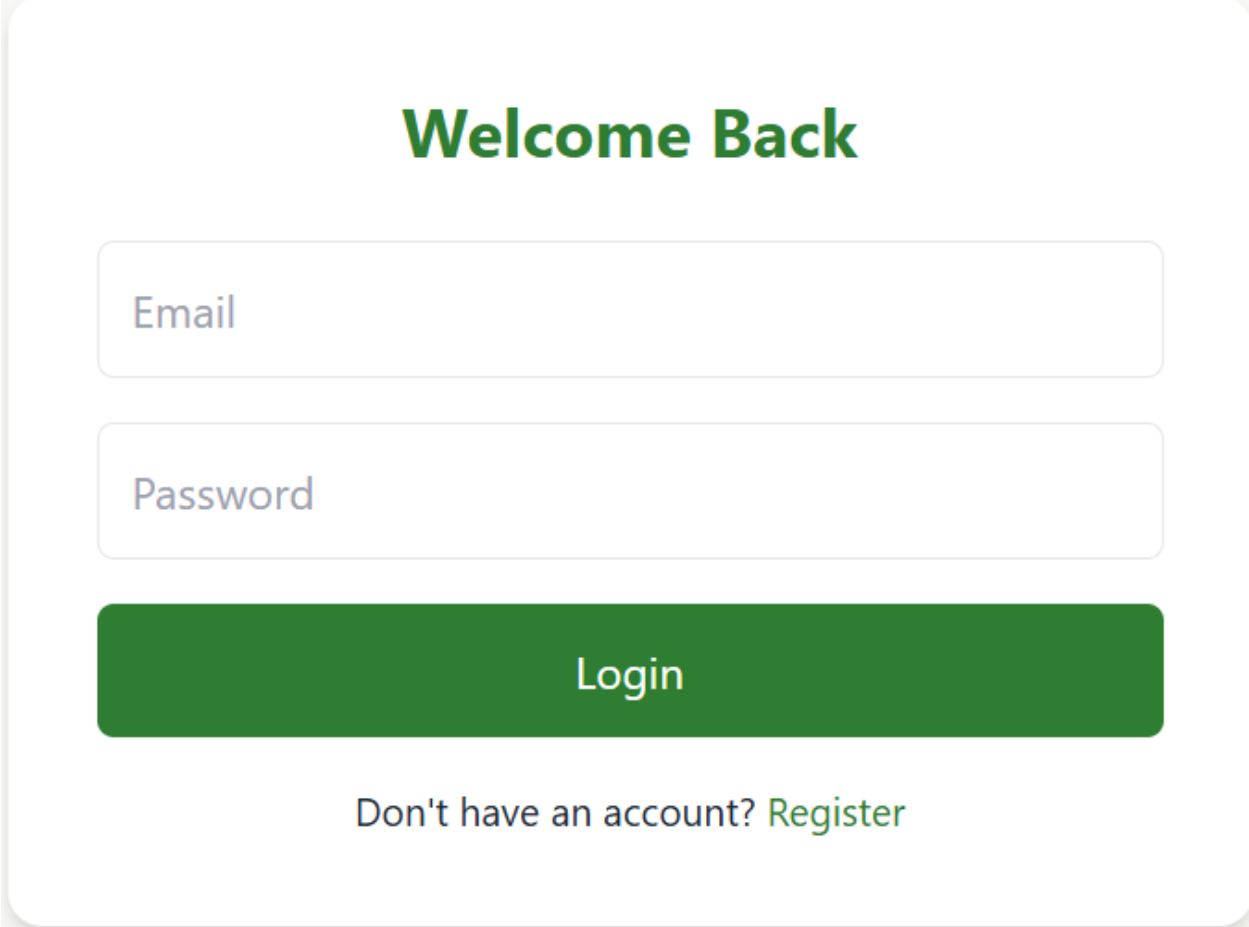
1. Landing page - It provides the users with information about the purpose of the application. It also navigates the options to register or log in.



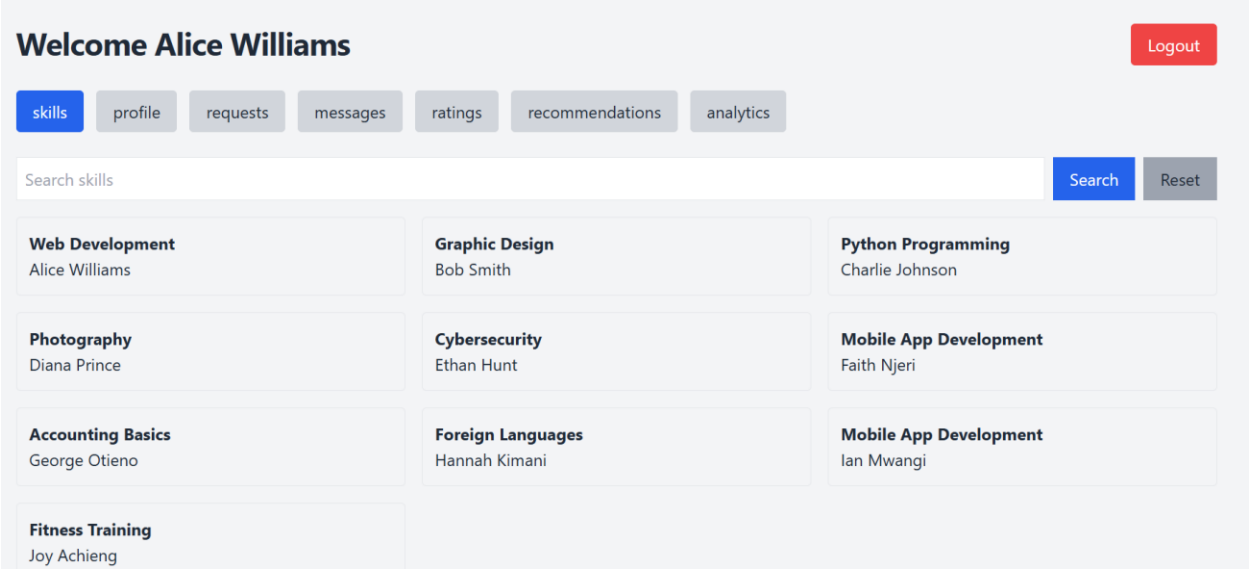
2. User registration - The user can create an account by providing a name, email and password.



3. User login - Authenticates users using stored credentials and generates a JWT token for session management.



4. User dashboard - Provides a personalized interface where users can view and add skills, manage their profile and view available skill listings.



5. Skill listing module - Users can create skill listings which include skill title, category, level and tags.

Title

Category

Beginner

Years experience

tags

Add Skill

Screenshots of code

1. React component (Dashboard.jsx)

```
import { useEffect, useState, useRef } from "react";
import { useNavigate } from "react-router-dom";
import api from "../api/axios";
import toast from "react-hot-toast";
import Skills from "./Skills";
import AddSkillForm from "./AddSkillForm";
import socket from "../socket";
import { Bar } from "react-chartjs-2";
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  Title,
  Tooltip,
  Legend,
} from "chart.js";

ChartJS.register(CategoryScale, LinearScale, BarElement, Title, Tooltip, Legend);

export default function Dashboard() {
  const navigate = useNavigate();
  const messageEndRef = useRef(null);
  const [user, setUser] = useState(null);
  const [profileSkills, setProfileSkills] = useState([]);
  const [marketplaceSkills, setMarketplaceSkills] = useState([]);
  const [requests, setRequests] = useState({ incoming: [], outgoing: [] });
  const [analyticsData, setAnalyticsData] = useState(null);
  const [recommendations, setRecommendations] = useState([]);
```

```
/* ---------------- Requests ---------------- */
useEffect(() => {
  if (activeTab !== "requests") return;
  async function loadRequests() {
    try {
      const res = await api.get("/requests");
      setRequests(res.data);
    } catch (err) {
      console.error(err);
    }
  }
  loadRequests();
}, [activeTab]);

/* ---------------- AI Recommendations ---------------- */
useEffect(() => {
  if (activeTab !== "recommendations") return;
  async function loadRecommendations() {
    try {
      const res = await api.get("/ai/recommendations");
      setRecommendations(res.data);
    } catch (err) {
      console.error(err);
    }
  }
  loadRecommendations();
}, [activeTab]);
```

```
/* ---------------- Ratings ---------------- */
useEffect(() => {
  if (activeTab !== "ratings") return;
  async function loadRatings() {
    try {
      const res = await api.get("/ratings/received");
      setRatings(res.data);
    } catch (err) {
      console.error(err);
    }
  }
  loadRatings();
}, [activeTab]);

/* ---------------- Analytics ---------------- */
useEffect(() => {
  if (activeTab !== "analytics") return;
  async function loadAnalytics() {
    try {
      const res = await api.get("/analytics");
      setAnalyticsData(res.data);
    } catch (err) {
      console.error(err);
    }
  }
  loadAnalytics();
}, [activeTab]);
```

```
/* ---------------- Socket Setup & Messages ---------------- */
useEffect(() => {
  if (!selectedExchange) return;

  // Join the exchange room
  socket.emit("join_exchange", selectedExchange);

  // Listen for incoming messages
  const handleReceiveMessage = (msg) => {
    setMessages((prev) => [...prev, msg]);
    // Scroll to bottom on new message
    setTimeout(() => {
      messageEndRef.current?.scrollIntoView({ behavior: "smooth" });
    }, 50);
  };

  socket.on("receive_message", handleReceiveMessage);

  // Handle reconnect: rejoin the room automatically
  const handleReconnect = () => {
    if (selectedExchange) {
      socket.emit("join_exchange", selectedExchange);
    }
  };
  socket.on("connect", handleReconnect);

  return () => {
    socket.off("receive_message", handleReceiveMessage);
```

```
    socket.off("connect", handleReconnect);
  };
}, [selectedExchange]);

/* --------------- Messages --------------- */
async function loadMessages(exchangeId) {
  try {
    const res = await api.get(`/messages/${exchangeId}`);
    setMessages(res.data);
    setSelectedExchange(exchangeId);
    setTimeout(() => {
      messageEndRef.current?.scrollIntoView({ behavior: "smooth" });
    }, 100);
  } catch {
    toast.error("Failed to load messages");
  }
}

async function sendMessage() {
  if (!newMessage.trim()) return;

  const msg = {
    exchangeId: selectedExchange,
    content: newMessage,
    sender_id: user.id,
  };
```

```
  // Emit to backend
  socket.emit("send_message", msg);

  // Persist in DB
  await api.post(`/messages/${selectedExchange}`, { content: newMessage });

  setNewMessage("");
}

/* ---------------- Skill Delete ---------------- */
async function handleDeleteSkill(id) {
  try {
    await api.delete(`/users/skills/${id}`);
    setProfileSkills((prev) => prev.filter((s) => s.id !== id));
    toast.success("Skill deleted");
  } catch {
    toast.error("Delete failed");
  }
}

function handleAddSkill(skill) {
  setProfileSkills((prev) => [skill, ...prev]);
}

/* ---------------- Logout ---------------- */
function handleLogout() {
  localStorage.removeItem("token");
```

```
    navigate("/login");
  }

  if (loading) {
    return <div className="p-10 text-center">Loading dashboard...</div>;
  }

  return (
    <div className="min-h-screen bg-gray-100 p-6">
      <div className="max-w-6xl mx-auto">
        {/* HEADER */}
        <div className="flex justify-between mb-6">
          <h1 className="text-3xl font-bold">Welcome {user?.name}</h1>
          <button
            onClick={handleLogout}
            className="bg-red-500 text-white px-4 py-2 rounded"
          >
            Logout
          </button>
        </div>

        {/* TABS */}
        <div className="flex gap-3 mb-6 flex-wrap">
          {[
            "skills",
            "profile",
            "requests",
```

```
        "messages",
        "ratings",
        "recommendations",
        "analytics",
      ].map((tab) => (
        <button
          key={tab}
          onClick={() => setActiveTab(tab)}
          className={`px-4 py-2 rounded ${
            activeTab === tab ? "bg-blue-600 text-white" : "bg-gray-300"
          }`}
        >
          {tab}
        </button>
      ))}
    </div>

    {/* MARKETPLACE */}
    {activeTab === "skills" && (
      <Skills
        currentUser={user}
        marketplaceSkills={marketplaceSkills}
        setMarketplaceSkills={setMarketplaceSkills}
      />
    )}

    {/* PROFILE */}
    {activeTab === "profile" && (
```

```
<div className="bg-white p-6 rounded shadow">
  <p>
    <b>Name:</b> {user.name}
  </p>
  <p>
    <b>Email:</b> {user.email}
  </p>
  <h3 className="mt-4 font-semibold">My Skills</h3>

  {profileSkills.map((skill) => (
    <div key={skill.id} className="border p-3 mt-2 flex justify-between">
      <div>
        <b>{skill.title}</b>
        <p>{skill.description}</p>
      </div>
      <button
        onClick={() => handleDeleteSkill(skill.id)}
        className="bg-red-500 text-white px-2 rounded"
      >
        Delete
      </button>
    </div>
  ))}

  <div className="mt-4">
    <AddSkillForm currentUser={user} onSkillAdded={handleAddSkill} />
  </div>
</div>
```

```
)}

{/* REQUESTS */}
{activeTab === "requests" && (
  <div className="bg-white p-6 rounded shadow">
    <h2 className="font-semibold mb-3">Incoming</h2>
    {requests.incoming.map((r) => (
      <div key={r.id}>{r.title}</div>
    ))}
    <h2 className="font-semibold mt-4 mb-3">Outgoing</h2>
    {requests.outgoing.map((r) => (
      <div key={r.id}>{r.title}</div>
    ))}
  </div>
)}

{/* MESSAGES */}
{activeTab === "messages" && (
  <div className="bg-white p-6 rounded shadow">
    {!selectedExchange && (
      <div>
        <h2 className="font-semibold mb-3">Select an Exchange</h2>
        {requests.outgoing.concat(requests.incoming).map((ex) => (
          <button
            key={ex.id}
            onClick={() => loadMessages(ex.id)}
            className="block border p-2 w-full text-left mb-2"
          >
```

```
                Exchange #{ex.id}
              </button>
            ))}
          </div>
        )}

        {selectedExchange && (
          <div>
            <div className="h-64 overflow-y-auto border p-3 mb-3">
              {messages.map((m) => (
                <div key={m.id} className="mb-2">
                  <b>{m.sender_name}</b>: {m.content}
                </div>
              ))}
              <div ref={messageEndRef} />
            </div>
            <div className="flex gap-2">
              <input
                value={newMessage}
                onChange={(e) => setNewMessage(e.target.value)}
                className="border flex-1 p-2"
              />
              <button
                onClick={sendMessage}
                className="bg-blue-600 text-white px-4"
              >
                Send
```

```
                    </button>
                  </div>
                </div>
              )}
            </div>
          )}

          {/* RATINGS */}
          {activeTab === "ratings" && (
            <div className="bg-white p-6 rounded shadow">
              {ratings.map((r) => (
                <div key={r.id} className="border p-3 mb-2">
                  ⭐ {r.score}
                  <p>{r.comment}</p>
                </div>
              ))}
            </div>
          )}

          {/* AI RECOMMENDATIONS */}
          {activeTab === "recommendations" && (
            <div className="bg-white p-6 rounded shadow">
              {recommendations.map((skill) => (
                <div key={skill.id} className="border p-3 mb-2">
                  {skill.title}
                </div>
              ))}
            </div>
          )}

          {/* ANALYTICS */}
          {activeTab === "analytics" && (
            <div className="bg-white p-6 rounded shadow">
              {analyticsData ? <Bar data={analyticsData} /> : <p>No analytics yet</p>}
            </div>
          )}
        </div>
      </div>
    );
}
```

```
const [ratings, setRatings] = useState([]);
const [messages, setMessages] = useState([]);
const [selectedExchange, setSelectedExchange] = useState(null);
const [newMessage, setNewMessage] = useState("");
const [activeTab, setActiveTab] = useState("skills");
const [loading, setLoading] = useState(true);

/* ---------------- Load Dashboard ---------------- */
useEffect(() => {
  async function loadDashboard() {
    try {
      const res = await api.get("/users/me");
      const userData = res.data;
      setUser(userData)   const userData: any
      setProfileSkills(userData.skills || []);

      const marketplace = await api.get("/skills");
      setMarketplaceSkills(marketplace.data);
    } catch (err) {
      console.error(err);
      toast.error("Failed to load dashboard");
    } finally {
      setLoading(false);
    }
  }
  loadDashboard();
}, []);
```

2. Backend (skills.routes.js)

```
import express from "express";
import pool from "../config/db.js";

const router = express.Router();

/*
=====================================================
GET MARKETPLACE LISTINGS
=====================================================
*/
router.get("/", async (req, res) => {
  try {
    const { q = "", tag = "" } = req.query;

    let query = `
      SELECT
        l.id AS id,
        s.id AS skillId,
        s.title,
        s.tags,
        u.id AS owner_id,
        u.name AS owner_name,
        COUNT(DISTINCT e.id) AS exchange_count
      FROM listings l
      JOIN skill_detail sd ON sd.id = l.skill_offered_detail_id
      JOIN skills s ON s.id = sd.skill_id
      JOIN users u ON u.id = sd.user_id
      LEFT JOIN exchanges e ON e.listing_id = l.id
      WHERE l.status = 'active'
```

```
    `;

    const params = [];

    if (q) {
      params.push(`%${q}%`);
      query += ` AND s.title ILIKE $${params.length}`;
    }

    if (tag) {
      params.push(`%${tag}%`);
      query += ` AND s.tags ILIKE $${params.length}`;
    }

    query += `
      GROUP BY l.id, s.id, u.id
      ORDER BY l.created_at DESC
    `;

    const { rows } = await pool.query(query, params);

    res.json(rows);
  } catch (err) {
    console.error("GET /skills error:", err);
    res.status(500).json({ message: "Failed to fetch listings" });
  }
});
```

```
/*
=====================================================
GET SINGLE LISTING DETAIL
=====================================================
*/
router.get("/:id", async (req, res) => {
  try {
    const query = `
      SELECT
        l.id AS id,
        s.id AS skillId,
        s.title,
        s.tags,
        u.id AS owner_id,
        u.name AS owner_name,
        sd.description,
        COUNT(DISTINCT e.id) AS exchange_count
      FROM listings l
      JOIN skill_detail sd ON sd.id = l.skill_offered_detail_id
      JOIN skills s ON s.id = sd.skill_id
      JOIN users u ON u.id = sd.user_id
      LEFT JOIN exchanges e ON e.listing_id = l.id
      WHERE l.id = $1
      GROUP BY l.id, s.id, u.id, sd.description
    `;

    const { rows } = await pool.query(query, [req.params.id]);

    if (!rows.length) {
```

```
      return res.status(404).json({ message: "Listing not found" });
    }

    res.json(rows[0]);
  } catch (err) {
    console.error("GET /skills/:id error:", err);
    res.status(500).json({ message: "Error fetching listing" });
  }
});

/*
=====================================================
DELETE LISTING
=====================================================
*/
router.delete("/:id", async (req, res) => {
  try {
    const listingId = req.params.id;

    if (!listingId || isNaN(listingId)) {
      return res.status(400).json({ message: "Invalid listing ID" });
    }

    await pool.query("DELETE FROM listings WHERE id = $1", [listingId]);

    res.json({ message: "Listing deleted successfully" });
  } catch (err) {
    console.error("DELETE /skills/:id error:", err);
```

```
    res.status(500).json({ message: "Failed to delete listing" });
  }
});

export default router;
```

3. Database

```sql
-- CLEAN RESET
DROP TABLE IF EXISTS connections;
DROP TABLE IF EXISTS messages;
DROP TABLE IF EXISTS ratings;
DROP TABLE IF EXISTS user_activity;
DROP TABLE IF EXISTS exchanges;
DROP TABLE IF EXISTS listings;
DROP TABLE IF EXISTS skill_detail;
DROP TABLE IF EXISTS skills;
DROP TABLE IF EXISTS users;

-- USERS
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- SKILLS
CREATE TABLE skills (
  id SERIAL PRIMARY KEY,
  title VARCHAR(100) NOT NULL UNIQUE,
  category VARCHAR(100),
  tags TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- USER SKILLS
CREATE TABLE skill_detail (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON DELETE CASCADE,
  skill_id INT REFERENCES skills(id) ON DELETE CASCADE,
  level VARCHAR(50),
  years_experience INT,
  description TEXT,
  active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (user_id, skill_id)
);

-- LISTINGS
CREATE TABLE listings (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON DELETE CASCADE,
  skill_offered_detail_id INT REFERENCES skill_detail(id) ON DELETE CASCADE,
  skill_requested_id INT REFERENCES skills(id),
  title VARCHAR(255),
  description TEXT,
  status VARCHAR(20) DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- EXCHANGES
CREATE TABLE exchanges (
  id SERIAL PRIMARY KEY,
  requester_id INT REFERENCES users(id) ON DELETE CASCADE,
```

```sql
  listing_id INT REFERENCES listings(id) ON DELETE CASCADE,
  status VARCHAR(20) DEFAULT 'pending',
  message_count INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- MESSAGES
CREATE TABLE messages (
  id SERIAL PRIMARY KEY,
  exchange_id INT REFERENCES exchanges(id) ON DELETE CASCADE,
  sender_id INT REFERENCES users(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_messages_exchange ON messages(exchange_id);

-- RATINGS
CREATE TABLE ratings (
  id SERIAL PRIMARY KEY,
  exchange_id INT REFERENCES exchanges(id) ON DELETE CASCADE,
  rater_id INT REFERENCES users(id),
  rated_user_id INT REFERENCES users(id),
  score INT CHECK (score BETWEEN 1 AND 5),
  comment TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(exchange_id, rater_id)
);
```

```sql
-- USER ACTIVITY
CREATE TABLE user_activity (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON DELETE CASCADE,
  action VARCHAR(50),
  skill_id INT REFERENCES skills(id),
  details JSONB,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- CONNECTIONS
CREATE TABLE connections (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON DELETE CASCADE,
  connected_user_id INT REFERENCES users(id) ON DELETE CASCADE,
  status VARCHAR(20) DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(user_id, connected_user_id)
);
```

4. API routes implementation

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";

import authRoutes from "./routes/auth.routes.js";
import userRoutes from "./routes/user.routes.js";
import skillRoutes from "./routes/skill.routes.js";
import exchangeRoutes from "./routes/exchange.routes.js";
import messageRoutes from "./routes/messages.routes.js";
import ratingRoutes from "./routes/ratings.routes.js";
import aiRoutes from "./routes/ai.routes.js";
import adminRoutes from "./routes/admin.routes.js";
import analyticsRoutes from "./routes/analytics.routes.js";
import requestsRoutes from "./routes/requests.routes.js";

dotenv.config();

const app = express();

/* ---------------- Middleware ---------------- */
app.use(cors());
app.use(express.json());

/* ---------------- Routes ---------------- */
app.use("/api/auth", authRoutes);
app.use("/api/users", userRoutes);
app.use("/api/skills", skillRoutes);
app.use("/api/exchange", exchangeRoutes);
app.use("/api/messages", messageRoutes);
```

```
app.use("/api/ratings", ratingRoutes);
app.use("/api/ai", aiRoutes);
app.use("/api/admin", adminRoutes);
app.use("/api/analytics", analyticsRoutes);
app.use("/api/requests", requestsRoutes);

/* ---------------- Health Check ---------------- */
app.get("/", (req, res) => {
  res.send("Ex-change API running 🚀");
});

export default app;
```

Technical details

Programming languages, frameworks and tools

Frontend:

1. React.js.
2. Tailwind CSS.
3. Vite.

Backend:

1. Node.js.
2. Express.js.

Database: PostgreSQL.
Authentication: JSON Web Tokens (JWT).
Development tools:

1. VS Code.
2. Postman.
3. pgAdmin.
4. GitHub.
5. Git.

System architecture

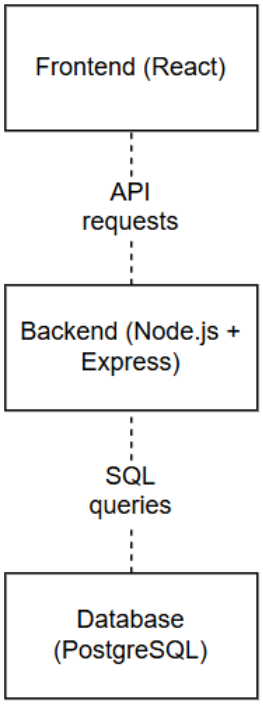


Figure1:three-tier system architecture.

Database schema

	table_name name	columns text
1	connections	id, status, created_at, connected_user_id, user_id
2	exchanges	status, updated_at, id, requester_id, listing_id, message_count, created_at
3	listings	id, title, status, description, created_at, skill_requested_id, skill_offered_detail_id, user_id
4	messages	exchange_id, created_at, sender_id, content, id
5	ratings	score, id, exchange_id, rater_id, rated_user_id, created_at, comment
6	skill_detail	level, description, created_at, active, user_id, years_experience, id, skill_id
7	skills	created_at, id, title, category, tags
8	user_activity	user_id, created_at, id, action, details, skill_id
9	users	created_at, name, email, password_hash, role, id

Figure 2: Database structure.

Testing and validation

Testing methods used

1. API testing using Postman.
2. Manual UI testing.
3. Database query validation.

Known limitations

The modules that are still in progress or under improvement.

Challenges and solutions

Problems encountered during coding.

1. Database relationships
   Relationships designs between tables (users, skills and exchanges) required normalization. The solution was to use an ER diagram to structure accurately before implementation.

2. API integration between frontend and backend
   Accurate communication between frontend and backend was difficult. Axios was used to efficiently handle API requests and responses.

3. Authentication and security
   Managing authentication sessions securely requires implementing token-based authentication. JWT authentication was implemented to securely manage user sessions.

Collaboration environment

GitHub

GitHub is used for:

1. Source code version control.
2. Code backups.
3. Tracking project development.

Repository Link: https://github.com/Chantal-Marissa-Pande/Ex-change

Development tools

1. Git for version control.
2. VS Code for development.
3. Postman for API testing.
4. pgAdmin for database management.

Name	Last commit message	Last commit date
..		
@socket.io/component-emitter	Backend functionality and socket implementation	yesterday
node_modules	Skill adding and skills page update	3 weeks ago
pgvector	Personalization and AI readiness	last month
sql	Backend functionality and socket implementation	yesterday
src	Backend functionality and socket implementation	yesterday
.env	Personalization and AI readiness	last month
README.md	Initial commit	last month
package-lock.json	Skill adding and skills page update	3 weeks ago
package.json	Skill adding and skills page update	3 weeks ago

Figure 3:GitHub Backend

Name	Last commit message	Last commit date
..		
public	Initial commit	last month
src	Backend functionality and socket implementation	yesterday
.env	Initial commit	last month
.gitignore	Initial commit	last month
README.md	Initial commit	last month
api.js	Initial commit	last month
eslint.config.js	Initial commit	last month
index.html	Initial commit	last month
package-lock.json	Backend functionality and socket implementation	yesterday
package.json	Backend functionality and socket implementation	yesterday
postcss.config.js	Initial commit	last month
tailwind.config.js	Initial commit	last month
vite.config.js	Admin dashboard creation	2 weeks ago

Figure 4:GitHub Frontend

Next steps

Features in progress

1. Exchange request workflow.
2. Messaging system between users.
3. Rating and review system.
4. AI-based skill matching.
5. Admin dashboard.
6. Analytics.

Timeline for completion:

Task	Completion time
Exchange request system	Week 9
Messaging module	Week 10
Rating system	Week 10
Admin dashboard	Week 11
Analytics	Week 11
AI skill matching	Week 11
Testing and debugging	Week 12

Percentage of work remaining

Current estimation:

1. Complete - 60%.
2. Remaining - 40%.

References

Ajira.(n.d.).Ajiradigital.go.ke.https://ajiradigital.go.ke/#/index

Calls for collaborative action to bridge Kenya's skills gap report – Kenya News Agency. (2025, November 6). Kenyanews.go.ke. https://www.kenyanews.go.ke/calls-for-collaborative-action-tobridge-kenyas-skills-gap-report/

Gebeya | Best of African Freelance Talent. (2023, February 17). Gebeya.com. https://gebeya.com/

https://jiji.co.ke/. (n.d.). Jiji.co.ke. https://jiji.co.ke/

Laura, M., Puerta, S., Johansson De Silva, S., & Rizvi, A. (2018). Kenya: Capturing Skills Requirements and Assessing Skills Gaps in the Modern Economy. https://documents1.worldbank.org/curated/en/295641532463546573/pdf/128788-

STEPESKenyaWorkingPaperFinalMayFULL.pdf
Skillshare: Teaching, Marketing & Promotions Sampler Course. (2024). Udemy. https://www.udemy.com/course/teachskillshare/?srsltid=AfmBOopbU1VcRBjgFoiX28VFHSMDR0iY_Pg1YqEZF3LyydxW36EUzM2z

Timerepublik. (n.d.). Timerepublik. https://timerepublik.com/
Upwork. (2024). Upwork | Hire Freelancers. Make things happen. Upwork. https://www.upwork.com/